

SI 313 WN26: Surveys Module Assignment

Part 1: Research Question and Hypothesis and

Part 2: Experimental Design

Nick Pisarczyk - 03/25/26

Part 1: Research Question and Hypothesis

Research Question:

On a university course portal like Canvas that doesn't have assignment reminders, do automated email and/or push notification reminders sent before incomplete assignment deadlines increase the number of assignments turned in on time? These reminders would be sent 48, 24, and 6 hours before the deadline, and would contain the name of the assignment, the due date/time for the assignment, and a link to the assignment page.

I would measure two outcomes here:

1. The frequency of on-time submissions (count of binary T/F)
2. The time-distance between the user's first engagement with the assignment page and the assignment deadline (Ex: assignment due April 2nd at 11:59. Opened April 1st at 11:59pm = 24h time distance. Opened March 30th 11:59 = 72h time distance)

The first outcome is an appropriate measure because it measures the end result of each case. I'm interested in whether more assignments are turned in on time as a result of these new reminders, so I need to measure the end result of whether assignments are turned in on time or not. The second is appropriate because it captures a behavioral response in the middle of the process. The new notifications would theoretically remind and encourage students to start assignments earlier (and therefore help them get these assignments done sooner), so measuring this distance captures whether the reminders are effective in this way.

Directional Hypothesis:

I expect that students who receive these reminders will show a higher rate of on-time submissions and a larger time-distance than students who receive no reminders. Considering Salganik's discussion of mechanisms in *Bit by Bit* section 4.4.3, I expect the treatment of sending notifications to work by changing an intermediate process rather than directly causing more on-time submissions; reminders help students remember deadline and might help battle procrastination by bugging them with a notification in their inbox or on their phone. The mechanism (memory support / curbing procrastination) should first show up as a larger time-distance between the assignment being opened and the due date, and then (if the mechanism holds) as a greater number of on-time submissions.

Part 2: Experimental Design

Randomization Unit

I'll be randomizing at the user level. In this case, randomizing which students receive the treatment in a given Canvas course. Randomizing at the user level keeps the design simple and gives you many independent units to study, and this method also matches how the reminder function operates (tells individuals based on their individual criteria). A tradeoff here might be contamination or spillover, where students might tell their classmates that they're getting reminders, potentially prompting other students to check Canvas when they otherwise wouldn't. Randomizing at a higher unit level, but you'd have to run it across many courses, and the course content, course format, and instructor could change the results wildly from course to course.

Design Type

This experiment will use between-subjects design, where students will be assigned the treatment or not be in the control group for the full span of the experiment. This design type makes the most sense, because I'm interested in the differences between students when they see the reminders right now. I'm not comparing them to their past selves, like is necessary in a within-subjects or mixed design.

While a mixed design might work here, it's too complex and time-intensive for the context of this experiment. Additionally, university students have the potential to change a lot year-to-year, or even semester-to-semester, adding confounding effects. Underclassmen might be getting acclimated to college life and miss more assignment deadlines in one semester, and do better the next semester. A student might experience a family event or have a relative pass away, meaning one semester is more difficult than it would be otherwise.

This design offers the most practical way to understand if this treatment is actually effective, and pretty effectively removes variables like comparing across different courses and professors, life events interfering with academics, and more.

Sample and Eligibility

My target population is University of Michigan students enrolled in participating Canvas courses. These courses must have graded assignments with clear deadlines, and must directly submit

their assignments through Canvas – e.g. a file upload, not a hand-in during lecture like we do for our reflection assignments in 313.

Inclusion Criteria:

- Enrolled University of Michigan student with an active Canvas account in the participating course
- At least 7 graded, non-participation-related / placeholder assignments
- Email and/or push notifications enabled on devices and in Canvas settings

Exclusion Criteria:

- Students with approved accommodations that require nonstandard deadlines - unless system can use each student's individualized due dates
- Assignments with no due date, rolling deadlines, participation assignment, other "placeholder" assignments
- Any students who opt out of research/notifications
- No auditors/instructors/TA Canvas accounts

Treatment Delivery

1. Setup:
 - a. Using the inclusion / exclusion criteria, select the eligible courses and assignments that notifications will be sent for. Assignments with online submissions and fixed due dates/times should be used.
 - i. For example, no placeholder attendance assignments should have notifications, only assignments like a problem set or paper submitted online should be included here.
 - b. Create an experiment table and randomly assign students the treatment or control condition. We'll call this variable "condition" and use the values "reminder" and "control"
 - c. Set up logs to track when each student receives reminders, when they open an assignment, and when they submit an assignment. If the assignment is not submitted by the due date or submitted after the due date, count that attempt as a "failure", and if it's submitted on time, count as a "success."
2. Screening / notification process:
 - a. This should run automatically, every 8 hours
 - b. Check Canvas for all eligible assignments with due dates in the next 57 hours
 - i. There must be an 9 hour buffer on top of the 48 hours so 48 hour reminder notifications can be sent
 - c. Check the submission status per student
 - i. Determine whether the assignment is submitted for each student / assignment pair

- d. Determine whether a reminder is needed
 - i. Between 48, 24, and 6 hour notifications, choose the reminder with the furthest distance to the due date that hasn't been sent yet
 - ii. Mark this notification as "should send" only if:
 1. The student is in the reminder condition
 2. Assignment is unsubmitted
 3. That specific reminder (48h / 24h / 6h before due date) hasn't already been sent
 - e. Queue the reminder
 - i. Create a notification with: student ID, assignment ID, course ID, the timing of the reminder, what channels it the reminder should be sent on (email/push), and the text of the reminder (including the link to the assignment and the due date with the timezone)
3. Notification delivery to students:
- a. Send to the student's enabled channels. If both email / push are enabled, send email first and push second.
 - b. Fill the message content:
 - i. Course name
 - ii. Assignment name
 - iii. Due date/time (with timezone)
 - iv. "Due in X hours" line (48h / 24h / 6h)
 - v. A direct link to the assignment page
 - c. Record the delivery. Log the timestamp, channel, and status of the sent notification (delivered or failed)
4. Implement stopping rules
- a. If the student ever submits an assignment, stop any reminders from being sent. If a submission exists and is found during the screening, skip any remaining reminders for that student/assignment pair.
 - b. If the deadline passes, don't send any reminders either.

Breakdown of the experience for each condition:

- Reminder: Students receive up to three reminders (48h / 24h / 6h before the deadline) for each eligible assignment they haven't submitted yet. Each reminder has a direct link to the assignment.
- Control: Students receive no automated reminders. They can see everything normally, but no added functionality with the reminders.

Measurement

Outcome 1: On-time submission frequency

- For each eligible assignment, create a binary on_time variable

```
if submission_timestamp <= due_timestamp:
    on_time = 1
else:
    on_time = 0
```

- Then summarize per student as both:
 - A count of the on_time submissions
 - The rate of on time submissions vs the total number of assignments past the deadline.
- Some validity threats
 - Different deadlines / individualized deadlines not applied in Canvas might misclassify as late
 - Instructor policies may differ, there may be some grace periods or audibles called by instructors that aren't shown in Canvas and miscount late vs on time
 - Any general technical issues with submissions. These systems aren't perfect.

Outcome 2: Time-distance between first engagement and the deadline (e.g. how early they start)

- For each assignment / student pair, define:
 - first_engagement_timestamp, the first timestamp after the assignment is posted when the student views the assignment page, or starts a submission attempt
 - time_distance = due_timestamp - first_engagement_timestamp (this value would be a datetime value)
- time_distance should be initialized to 0 in the case that the assignment isn't ever opened / submitted.
- Some validity threats:
 - Opening the assignment might not reflect a full understanding of the assignment. But since some assignments and classes use external resources like google doc instructions for their assignment details, this format would make the most sense.
 - Depending on the course and their working style, students might work on other platforms like Google Docs and not open the assignment page until they're about to submit. This might underestimate the time-distance.